



Servlets

“

Una clase Java que recibe una petición HTTP, la procesa — persistencia, lógica de negocio... — y devuelve una respuesta al cliente. El **controlador** sobre el que se construye toda la capa web de Jakarta EE.

¿Qué resuelve?

Antes de los servlets, generar HTML dinámico en Java era artesanal. El servlet estandariza el ciclo **petición** → **proceso** → **respuesta** y delega su gestión al contenedor (Tomcat, WildFly...).

¿Dónde vive?

Dentro de una **webapp**, desplegado en un contenedor de servlets. El contenedor crea, gestiona y destruye sus instancias — el desarrollador no llama `new` sobre un servlet.

Por qué importa hoy

Spring MVC funciona sobre un único servlet — el `DispatcherServlet`. Entender esta pieza es entender qué hay debajo de cualquier framework Java web.

CONTENIDO

02 ¿Qué es un Servlet?

04 Patrón MVC con Servlets

06 Métodos HTTP soportados

08 Configuración y primer Servlet

03 Estructura de una WebApp

05 Request y Response

07 Ciclo de vida y jerarquía

OBJETIVO DE HOY

Entender el ciclo de vida y el flujo **petición** → **respuesta** de un Servlet antes de escribir el primero en la práctica de esta clase.

¿Qué es un Servlet?

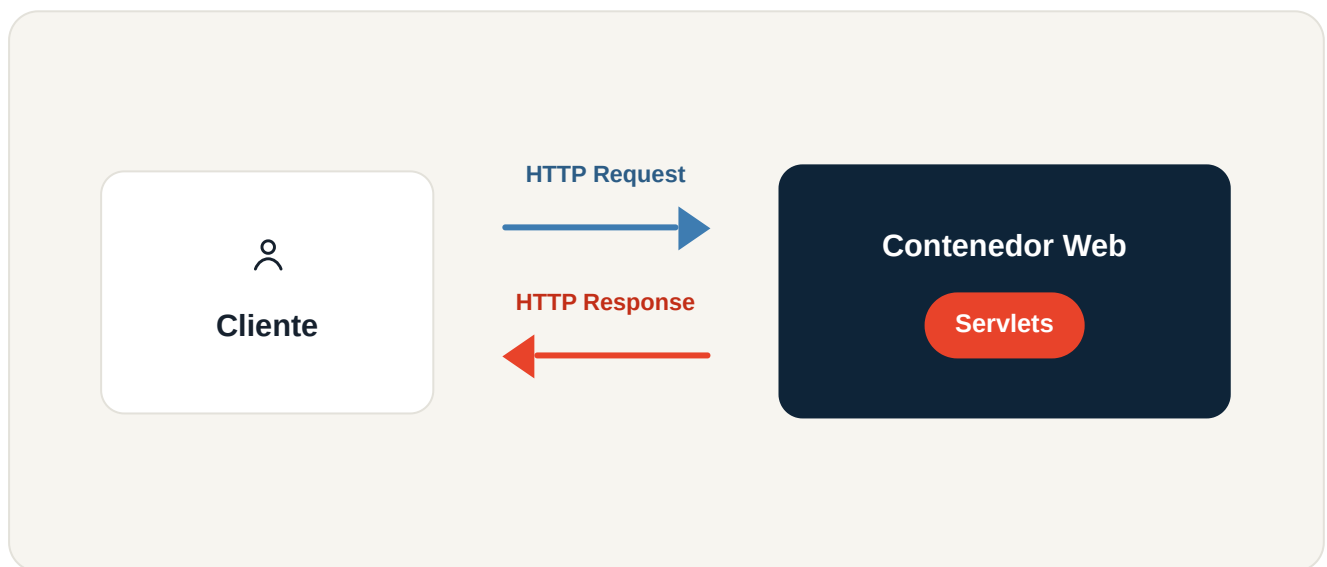
Uno de los **conceptos fundamentales** de la programación web en Java.

Es una **clase y objeto Java** utilizado para implementar una página web dinámica con características HTTP de petición y respuesta.

Se crea, se almacena y tiene un **ciclo de vida controlado por el contenedor** de servlets — nunca lo instancias tú con `new`.

Se mapea a una **ruta URL** a la que se accede desde un cliente web.

En el objeto `HttpServletRequest` llegan los parámetros; el servlet los procesa y devuelve la respuesta al cliente, o redirige a otro recurso.



El servlet nunca se llama directamente desde el cliente: siempre pasa primero por el **contenedor**, que decide qué instancia usar y qué método invocar.

Características clave



Singleton

una instancia por servlet



Multihilo

un hilo nuevo por petición



En memoria

vive mientras el servidor está activo



Portable

funciona en cualquier contenedor Jakarta EE

Dónde vive: estructura de una WebApp

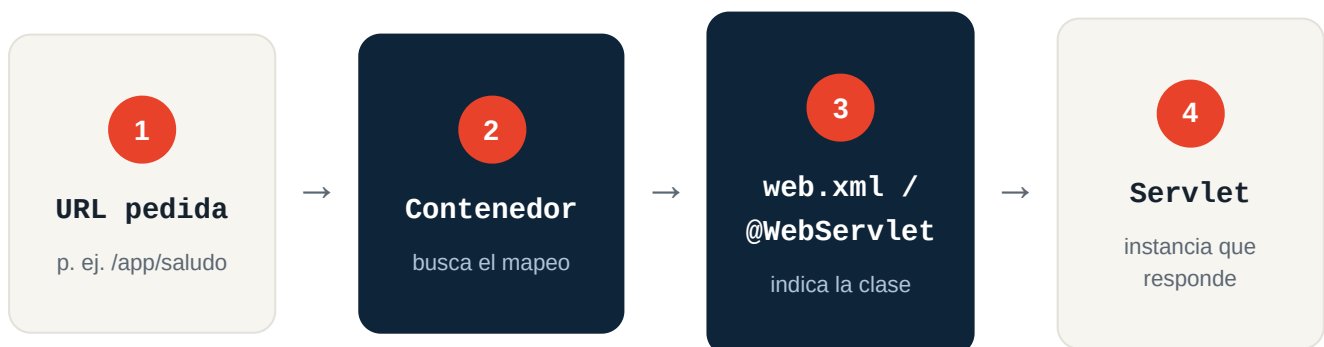
Un servlet no vive suelto: forma parte de una **aplicación web** con una estructura de carpetas estándar dentro del contenedor.

w	webapp/	Carpeta raíz de la aplicación desplegada	RAÍZ
w	WEB-INF/	Protegida — no accesible directamente desde el navegador	PRIVADA
l	lib/	Librerías <code>.jar</code> de terceros	LIBRERÍAS
c	classes/	Tus <code>.class</code> compilados — aquí viven los Servlets	COMPONENTES
x	web.xml	Descriptor de despliegue — configuración de la app	CONFIGURACIÓN
h	index.html	Páginas estáticas — accesibles directamente desde el cliente	PÁGINAS

El archivo web.xml

Es el **descriptor de despliegue** (deployment descriptor): configura el mapeo de servlets, filtros, listeners y parámetros de inicialización. Su uso ya no es obligatorio — depende de la versión de Servlet/Jakarta EE que uses (ver página 8).

Cómo se resuelve una petición



Todo lo que esté dentro de **WEB-INF/** está protegido: el cliente jamás puede pedirlo por URL directamente — solo lo usa el contenedor.

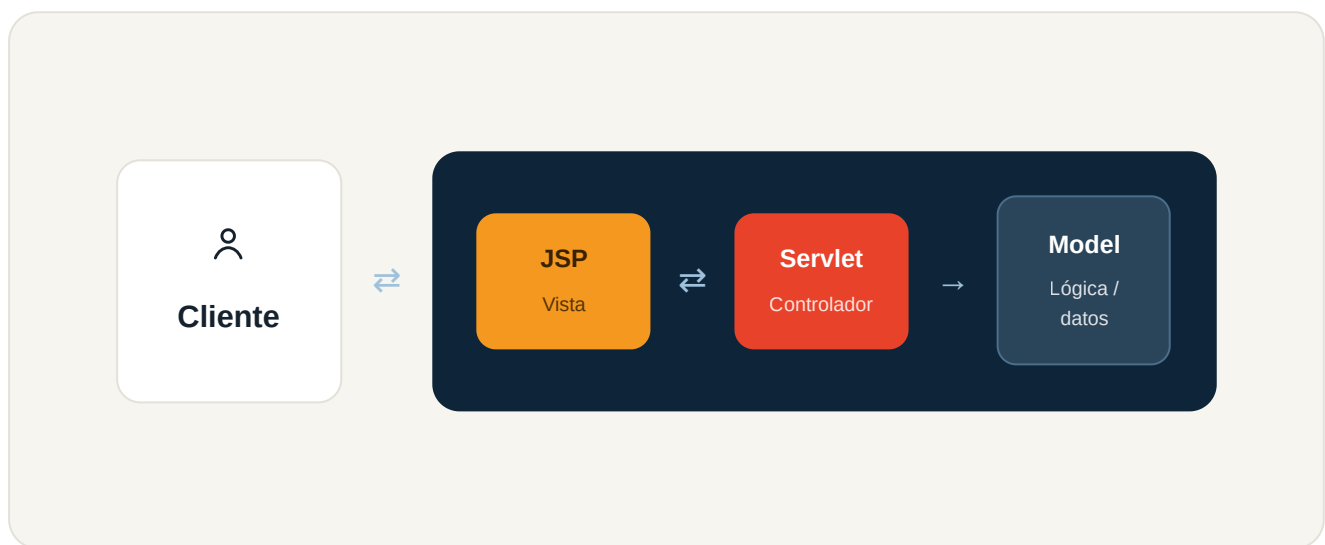
Patrón MVC con Servlets

Con los servlets podemos implementar la arquitectura MVC, separando responsabilidades:

Controlador → Servlet (puede hacer también de vista con HTML embebido, pero no es lo recomendado: no es mantenible).

Modelo → lógica de negocio (persistencia de datos, un servicio...).

Vista → JSP, por ejemplo (presentación).



Es la misma separación de roles que vimos en la guía de **Jakarta EE**: el Servlet es, exactamente, el **Controlador** de ese MVC mínimo.

Del Servlet a la vista



```
// desde el doGet() del Servlet – pasa datos y delega el HTML al JSP
request.setAttribute("productos", lista);

RequestDispatcher rd = request.getRequestDispatcher("/vista.jsp");
rd.forward(request, response);
```

Request y Response

¿Qué es un request?

Información enviada desde un cliente (navegador, app móvil...) hacia el servidor (Tomcat, WildFly, Payara, GlassFish...).

Datos ingresados y enviados por el usuario.

Método HTTP → `GET` (url) o `POST` (formulario).

Cabeceras HTTP (headers, metadatos del cliente).

¿Qué es un response?

Información enviada al cliente desde el servidor.

Texto (html, plain, json, xml) o datos binarios (imágenes, pdf, vídeos).

HTTP headers (content-type, status...), cookies, etc.

De la petición a la respuesta



Métodos habituales

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) {
    String nombre = request.getParameter("nombre"); // leer el request
    String agente = request.getHeader("User-Agent");

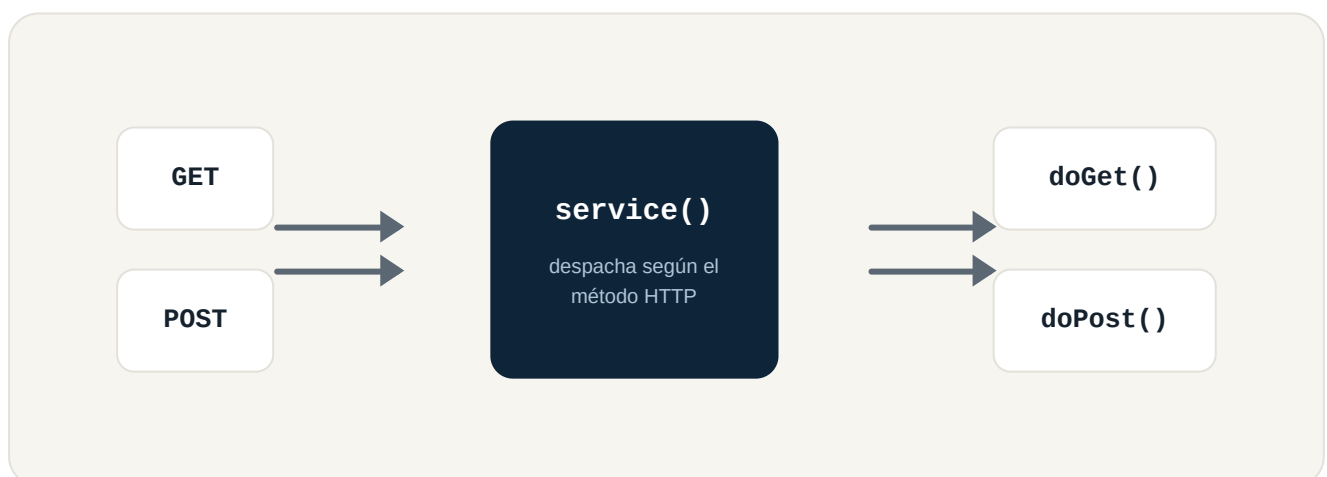
    response.setContentType("text/html;charset=UTF-8"); // preparar el response
    PrintWriter out = response.getWriter();
    out.println("Hola, " + nombre);
}
```

Métodos HTTP soportados

El API Servlet soporta 7 métodos HTTP. En un servlet, los más usados son `doGet` y `doPost` — y en una API REST, los que aparecen destacados abajo.

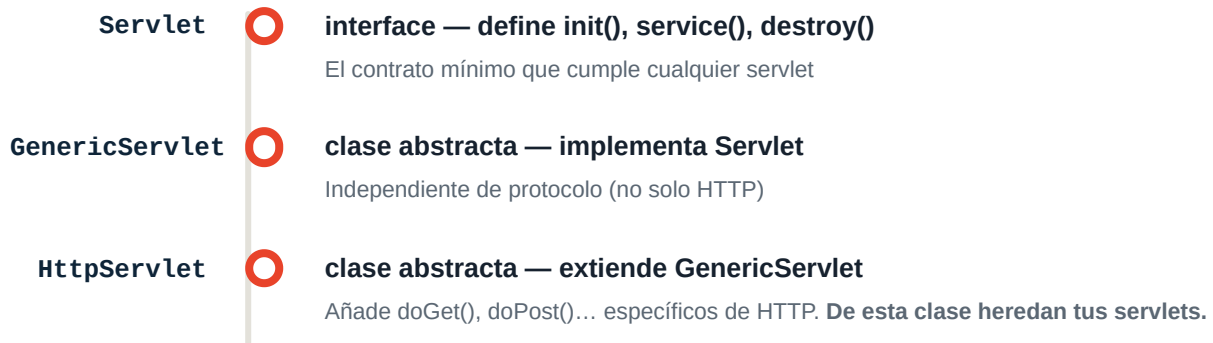
MÉTODO	CUÁNDO SE USA
doGet frecuente	Recuperar un recurso — parámetros van en la URL. El más usado en un servlet.
doPost frecuente	Enviar datos de un formulario o crear un recurso — parámetros en el cuerpo de la petición.
doPut	Actualizar (reemplazar) un recurso existente — muy usado en API REST.
doDelete	Eliminar un recurso — muy usado en API REST.
doHead	Como GET pero solo cabeceras, sin cuerpo de respuesta.
doOptions	Consulta qué métodos admite el recurso.
doTrace	Depuración: devuelve la petición tal como llegó al servidor.

Quién reparte cada petición



El método `service()` nunca debemos sobreescribirlo en `HttpServlet`: es el encargado de invocar a `doGet` o `doPost` por nosotros.

Jerarquía de clases



Ciclo de vida de un Servlet



Buenas prácticas

Usa siempre **variables locales** dentro de `doGet` / `doPost` — son propias de cada hilo, nunca se comparten.

Evita atributos de instancia mutables en la clase del servlet: todos los hilos los verían a la vez.

Si de verdad necesitas estado entre peticiones de un mismo usuario, usa `HttpSession`; si es global, un servicio con acceso sincronizado o la base de datos.

web.xml vs. anotaciones

HASTA SERVLET 2.5

Todo en web.xml

- Servlets, rutas, seguridad y contexto — todo declarado en XML.
- Archivo grande y repetitivo en apps con muchos servlets.

DESDE SERVLET 3.0+

web.xml + anotaciones

- `@WebServlet` mapea la ruta directamente en la clase
- `@WebFilter` y `@Security` reducen aún más el XML
- web.xml queda para lo que de verdad conviene centralizar (p. ej. seguridad global)

Tu primer Servlet



```
@WebServlet("/saludo")
public class SaludoServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h1>Hola, Servlet!</h1>");

    }
}
```



Con esto tienes las piezas del **controlador fundamental** de Jakarta EE: cómo vive, cómo recibe y responde, y cómo se configura. Es la base para escribir el primer Servlet en la práctica de hoy.